```
  (current, currentIndex) ← (pairs[index],
index)                                #1
  while currentIndex < firstLeafIndex(pairs) do
                  #2
     (child, childIndex) ←
highestPriorityChild(currentIndex)                #3
    if child.priority > current.priority
then                                  #4
      pairs[currentIndex] ←
pairs[childIndex]                                 #5
     currentIndex ← childIndex
    else
                #6
      break
  pairs[currentIndex] ← current
                                          #7
```

#1 Start at the index passed as argument (by default at the first element of the array A)

#2 Leaves have no children, so can't be pushed down anymore. `firstLeafIndex` returns the index of the first leaf in the heap: if `D` is the branching factor, then it's `|pairs| / D`

#3 We need to identify current node's children with the highest priority, because that will be the only element that is safe to move up without breaking heap properties

#4 If the highest priority child has higher priority than the current element `p`...

#5 ... then moves that child one position up (implicitly `p` goes one down)

#6 Otherwise, we have found the **right** place for `p`

#7 Arrived at this line, `currentIndex` is the right position for `p`